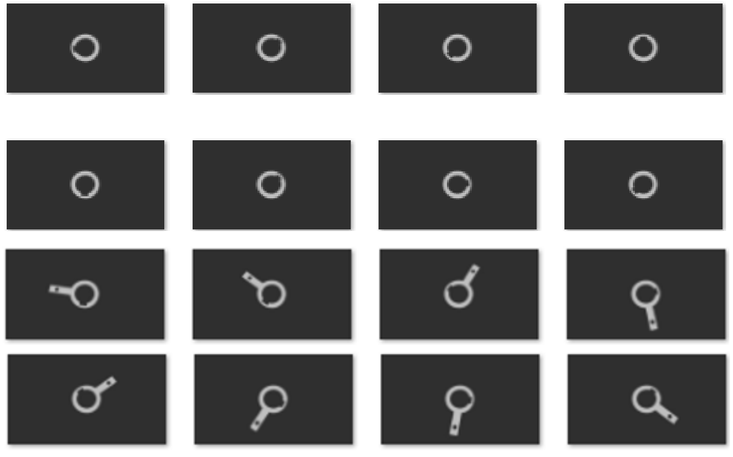
Реализация бинарной классификации для задач сортировки.

26.02.2026

Задача обнаружения движения

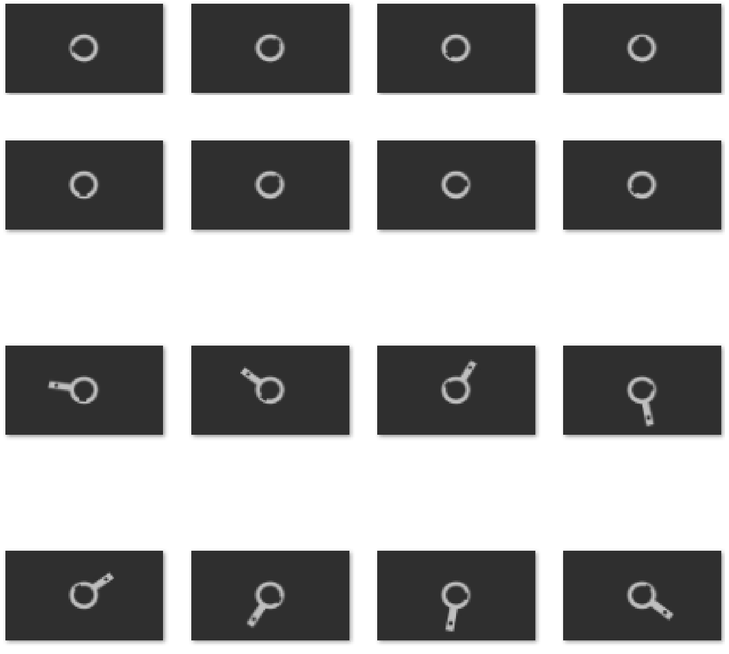
- Создать фон
- Загрузить 2 модели из urdf
- Создать камеру (вид сверху)
- Сгенерировать 500 изображений для каждой модели (на каждом кадре угол поворота задавать через random)

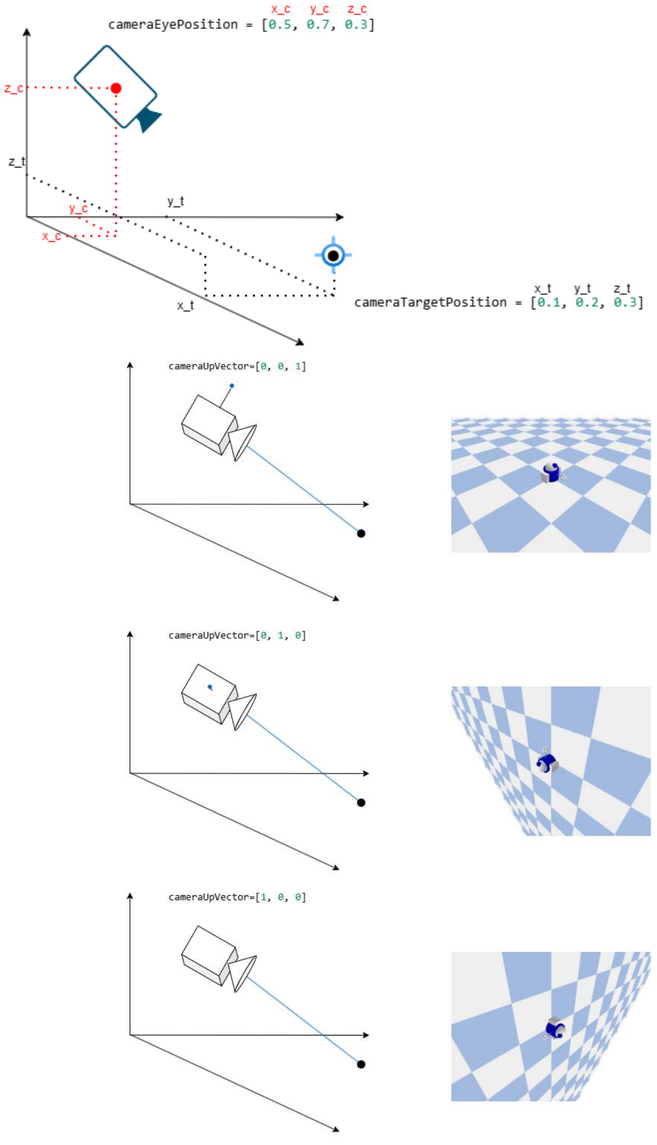


MODEL1 = "C:/urdf/mufta/1variant/urdf/1variant.urdf" MODEL2 = "C:/urdf/mufta/2variant/urdf/2variant.urdf"	Загружаем модели для двух вариантов деталей

Задача обнаружения движения

- Создать фон
- Загрузить 2 модели из urdf
- Создать камеру (вид сверху)
- Сгенерировать 500 изображений для каждой модели (на каждом кадре угол поворота задавать через random)





width = 1920
height = 1080
fy = 1796.1276552044753

near = 0.001
far = 3

fov = f (height, fy)
aspect = f (width, height)

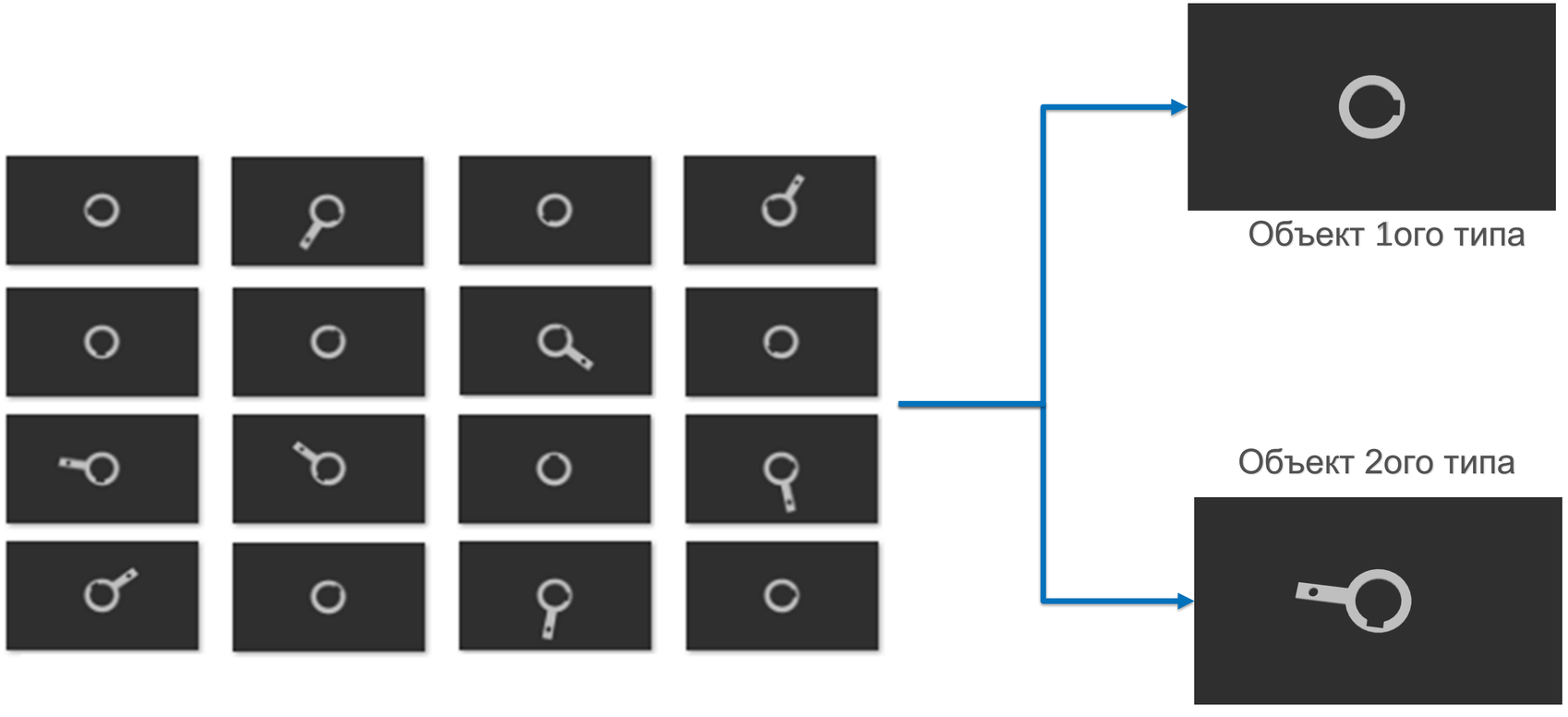
projection_matrix = p.computeProjectionMatrixFOV(fov, aspect, near, far)

eye = np.array([x.x, x.x, x.x])
target = np.array([x.x, x.x, x.x])
up_world = np.array([x.x, x.x, x.x])

```
view_matrix = p.computeViewMatrix(
    cameraEyePosition=eye,
    cameraTargetPosition=target,
    cameraUpVector=up_world
)
```

Задача обнаружения движения

На этом занятии Ваша задача реализовать бинарную классификацию, то есть научиться извлекать из изображения информацию о том: какого типа объект попал в кадр. По результатам работы нужно написать отчет и оценить насколько точно работает Ваша бинарная классификация.

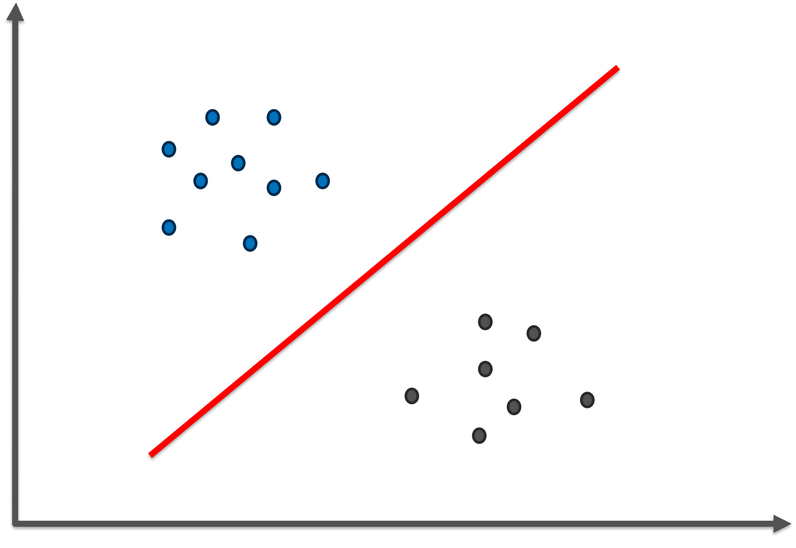


Задача работы

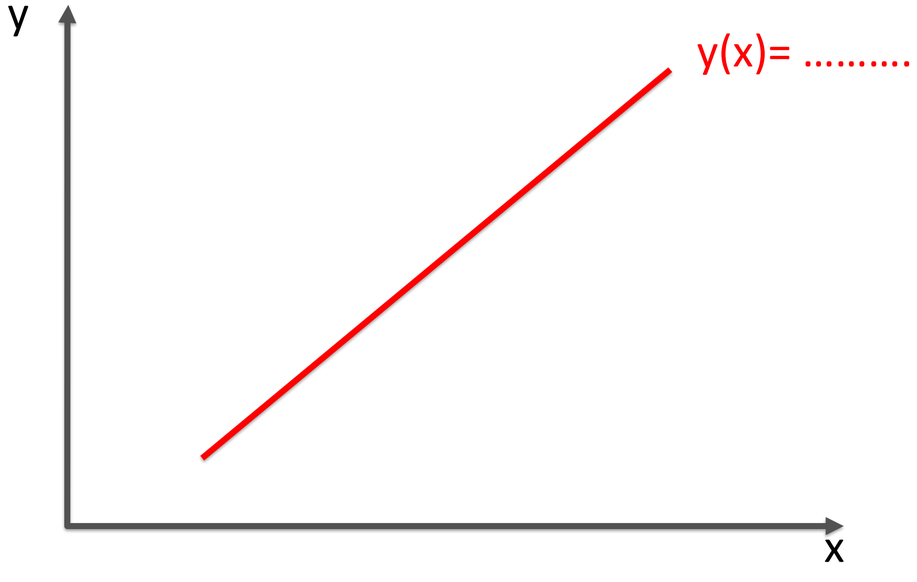
Нам бы хотелось, как можно меньше ресурсов тратить на получение требуемого результата, поэтому предварительно мы хотим максимально упростить входное изображение.

Задача работы

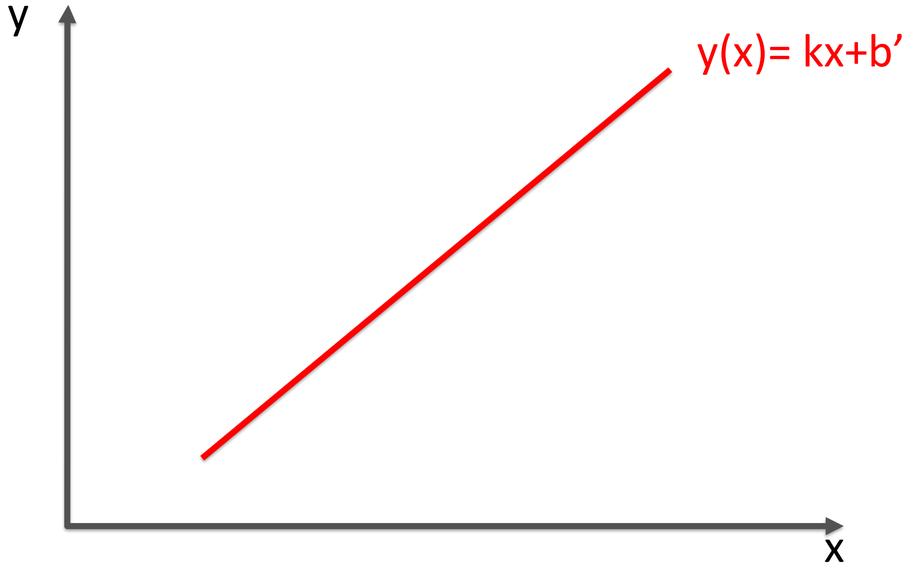
Теперь обсудим математику, который лежит в основе бинарной классификации с использованием выпуклой оптимизации. Хорошее учебное пособие для самостоятельного изучения – Выпуклая оптимизация: учебное пособие / Е. А. Воронцова, В92 Р.Ф.Хильдебранд,А.В.Гасников, Ф. С. Стонякин.–Москва: МФТИ, 2021.–364 с.–ил.–21 см.–Библиогр.: 326 назв. ISBN 978-5-7417-0776-0



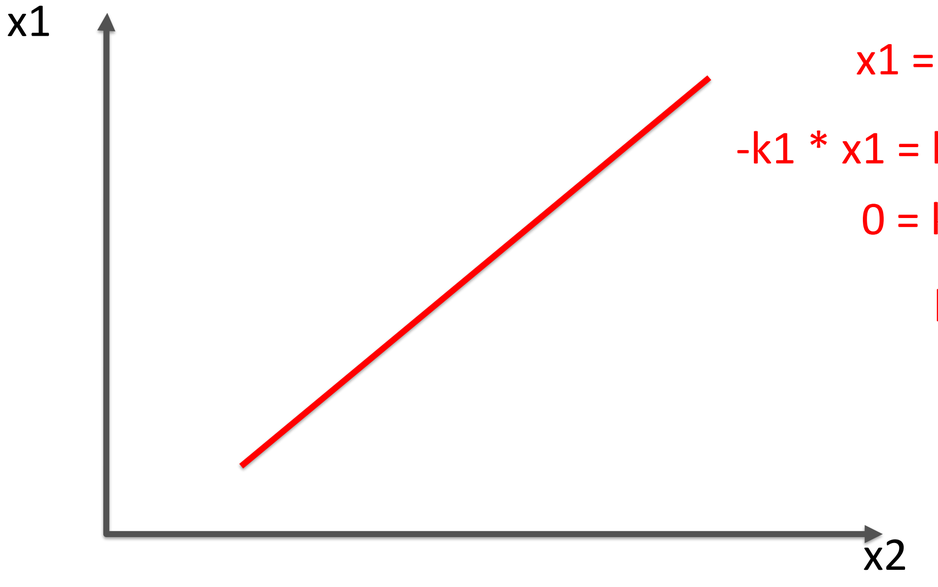
Задача работы



Задача работы



Задача работы



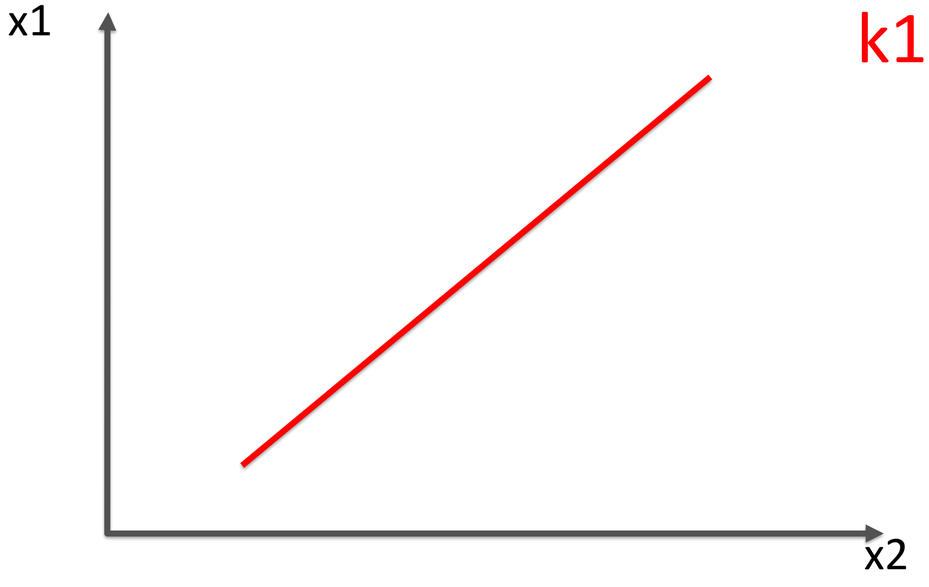
$$x1 = k * x2 + b'$$

$$-k1 * x1 = k2 * x2 + b$$

$$0 = k1 * x1 + k2 * x2 + b$$

$$k1 * x1 + k2 * x2 + b = 0$$

Задача работы

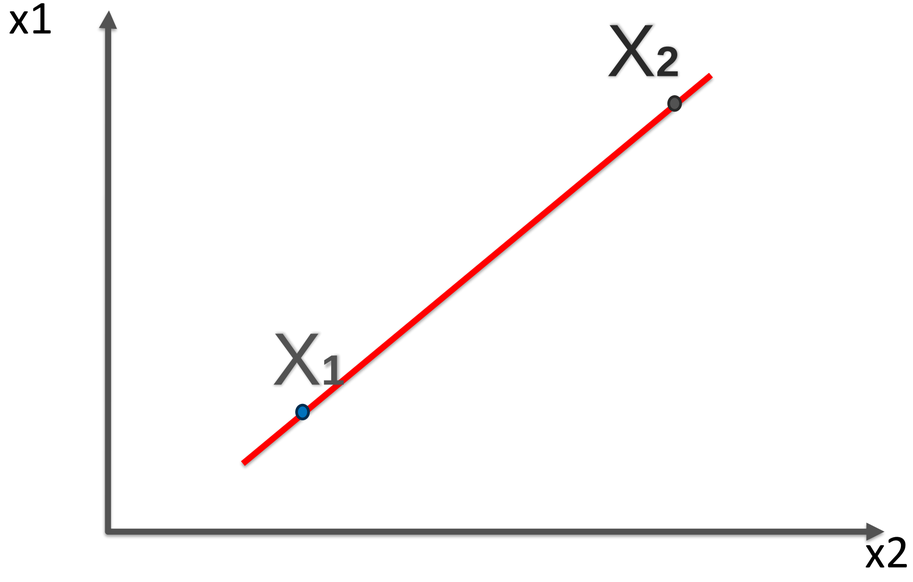


k1 * x1 + k2 * x2 + b =0

$$[k_1 \;\; k_2]\begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + b = 0$$

$$K^T \cdot \mathrm{X} + b = 0$$

Задача работы



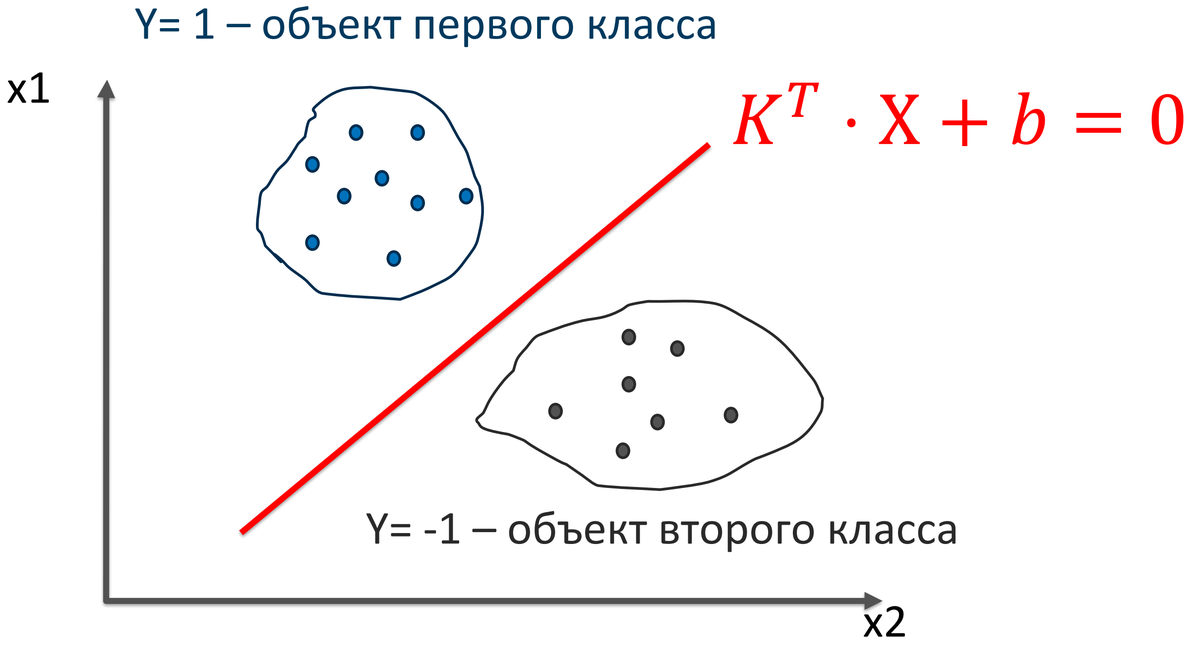
$X_1, X_2 \in$ гиперплоскости

$$\begin{cases} K^T \cdot \mathrm{X}_1 + b = 0 \\ K^T \cdot \mathrm{X}_2 + b = 0 \end{cases}$$

$$K^T \cdot (\mathrm{X}_1 - \mathrm{X}_2) = 0$$

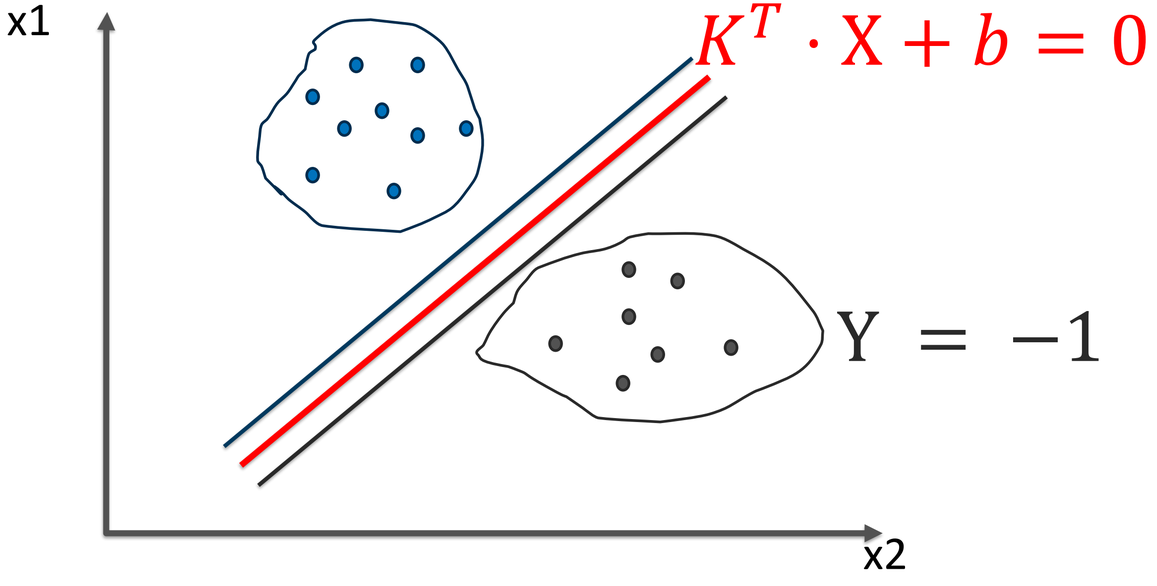
Векторы являются перпендикулярными тогда и только тогда, когда их скалярное произведение равно нулю

Задача работы



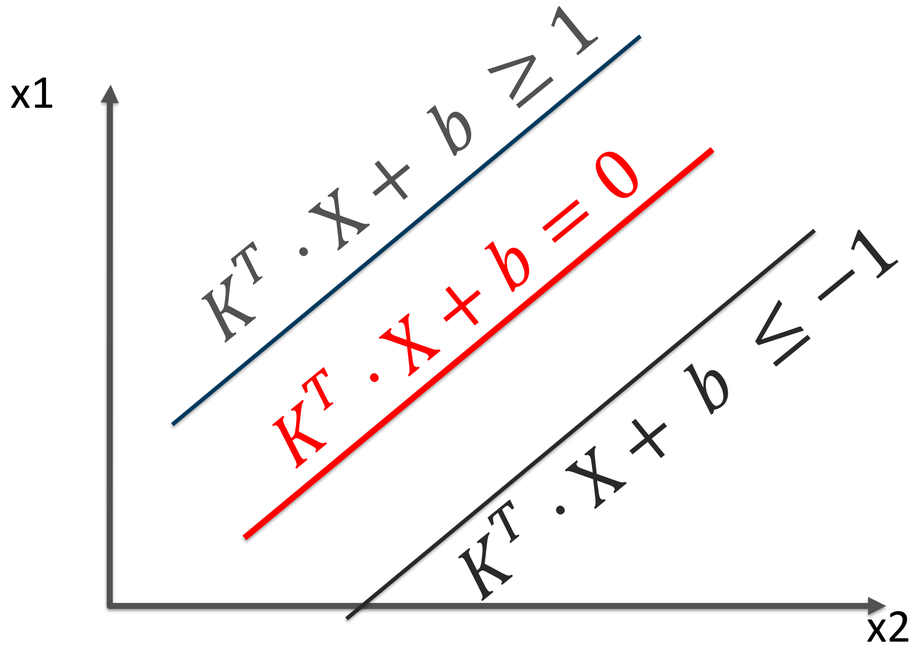
Задача работы

$$Y = 1 \iff K^T \cdot X + b > 0$$



$$Y = -1 \iff K^T \cdot X + b < 0$$

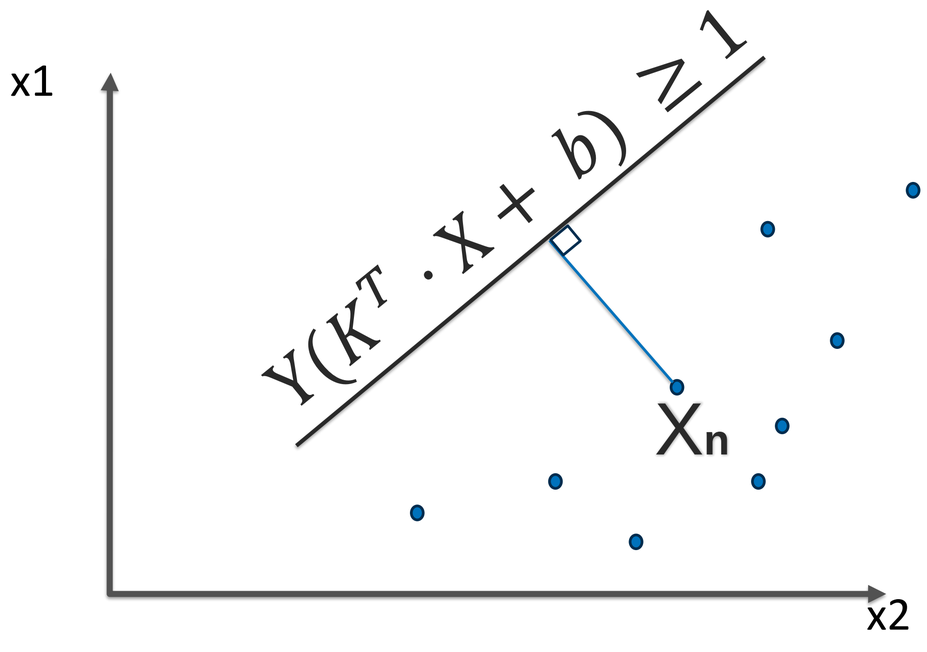
Задача работы



$$K^T \cdot \mathrm{X} + b \geq 1$$
$$K^T \cdot \mathrm{X} + b \leq -1$$

$$\mathrm{Y}(K^T \cdot \mathrm{X} + b) \geq 1$$

Задача работы



$$Y(K^T \cdot X + b) \geq 1$$

Если задано уравнение плоскости Ax + By + Cz + D = 0, то расстояние от точки M до плоскости можно найти, используя следующую формулу:

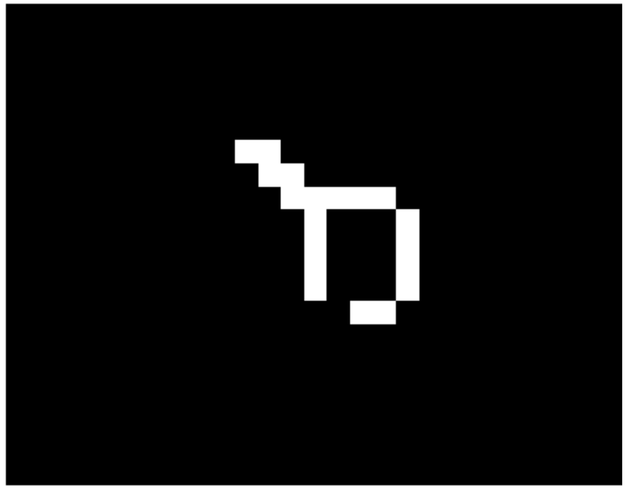
$$d = \frac{|AM_x + BM_y + CM_z + D|}{\sqrt{A^2 + B^2 + C^2}}$$

$$d(X_n) = \frac{|Y(K^T \cdot X + b|}{\sqrt{k_1{}^2 + k_2{}^2 + \ldots + k_n{}^2}} = \frac{|Y(K^T \cdot X + b|}{\|K^T\|}$$

Задача работы

```python
def load_dataset(labels_file):
    X = []
    Y = []

    with open(labels_file, "r", encoding="utf-8") as f:
        for line in f:
            fname, label = line.strip().split()
            label = int(label)

            img = cv2.imread(os.path.join(IMG_DIR, fname), cv2.IMREAD_GRAYSCALE)
            img = (img > 0).astype(np.float32)

            x = img.flatten()
            y = 1 if label == 1 else -1

            X.append(x)
            Y.append(y)

    return np.array(X), np.array(Y)
```

$$Y(K^T \cdot X + b) \geq 1$$



```
X:
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 1.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.
 0. 0. 0. 0. 0. 0.]
Y: -1
```

Задача работы

```
X_train, y_train = load_dataset(LABELS_FILE)

n_samples, n_features = X_train.shape

K = cp.Variable(n_features)
b = cp.Variable()

constraints = [y_train[i] * (X_train[i] @ K + b) >= 1
    for i in range(n_samples)]

objective = cp.Minimize(cp.norm(K, 2))


problem = cp.Problem(objective, constraints)
problem.solve()
print("Статус:", problem.status)
```

$$\mathrm{Y}(K^T \cdot \mathrm{X} + b) \geq 1$$

RUSSIAN SCIENCE IS FASCINATING!